Slides
and code

Spring AI

What's new, and putting AI to work in your applications

Dan Vega

Spring Developer Advocate · Broadcom

ABOUT ME



Dan Vega

Spring Developer Advocate · Broadcom

25 years building software, still shipping every week

Java Champion

Recognized for community contribution

Author

www.fundamentalsofswe.com

Conference speaker

Talks on Spring, Java and now AI

danvega.dev

Newsletter, blog and all the code

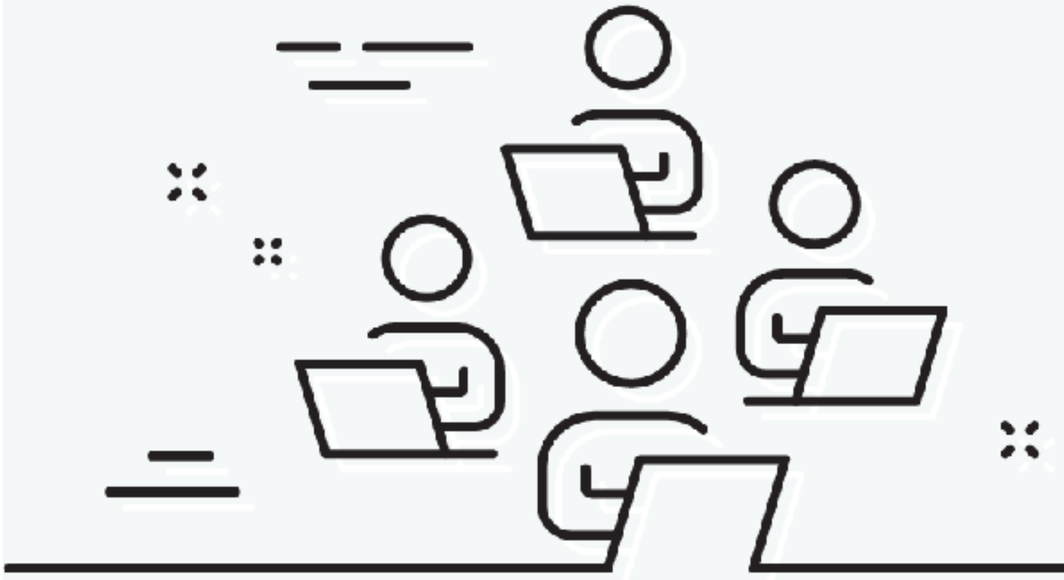
YouTube · Spring, Java & AI

Podcast · Spring Office Hours

Author · Articles, Courses and a new Book

spring

OFFICE HOURS







https://www.springofficehours.io

Today

Agenda

01 What companies use gen AI for

02 What's new in Spring AI 2

03 MCP: your services, ready for agents

04 Decision models: Jev and TypeSafe

05 Recap and Q&A

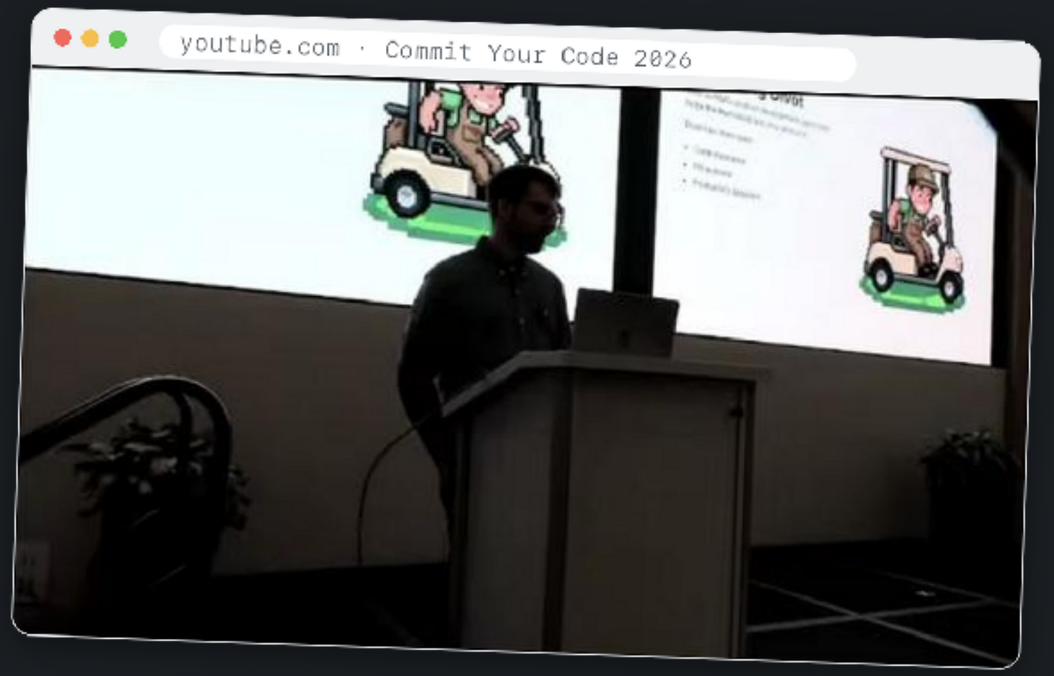


Part one

01

What companies use gen AI for

Treat AI like an intern.



Divot, the PGA of America's AI agent

The intern ladder

More leash as it earns trust

1

Read

Find, summarize

2

Draft

A person approves

3

Suggest

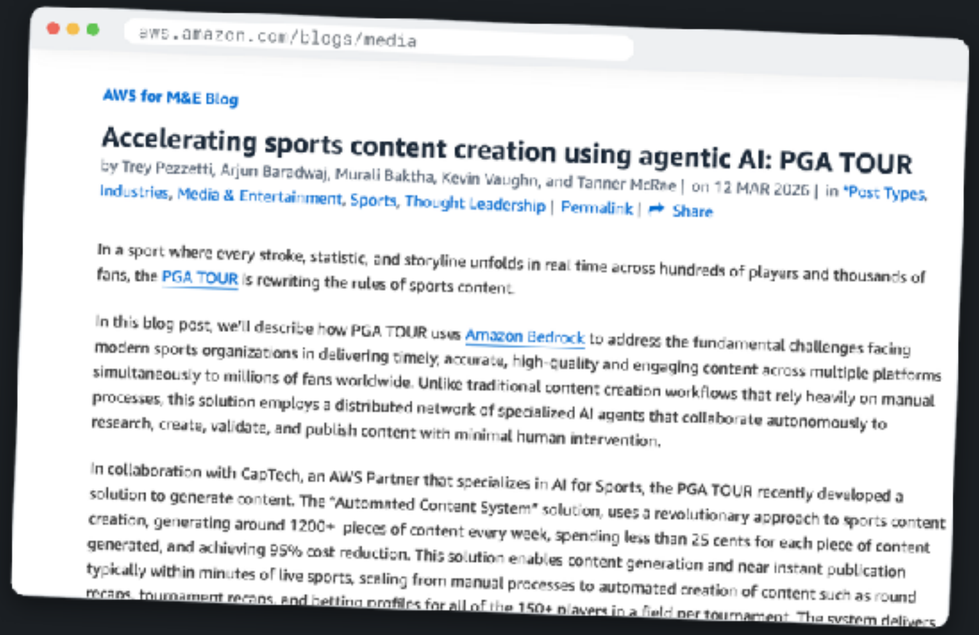
A person decides

4

Act

Narrow, gated

At scale



PGA TOUR

1,200+

Pieces of content a week, under 25 cents each

85%

of bank gen AI use cases are internal

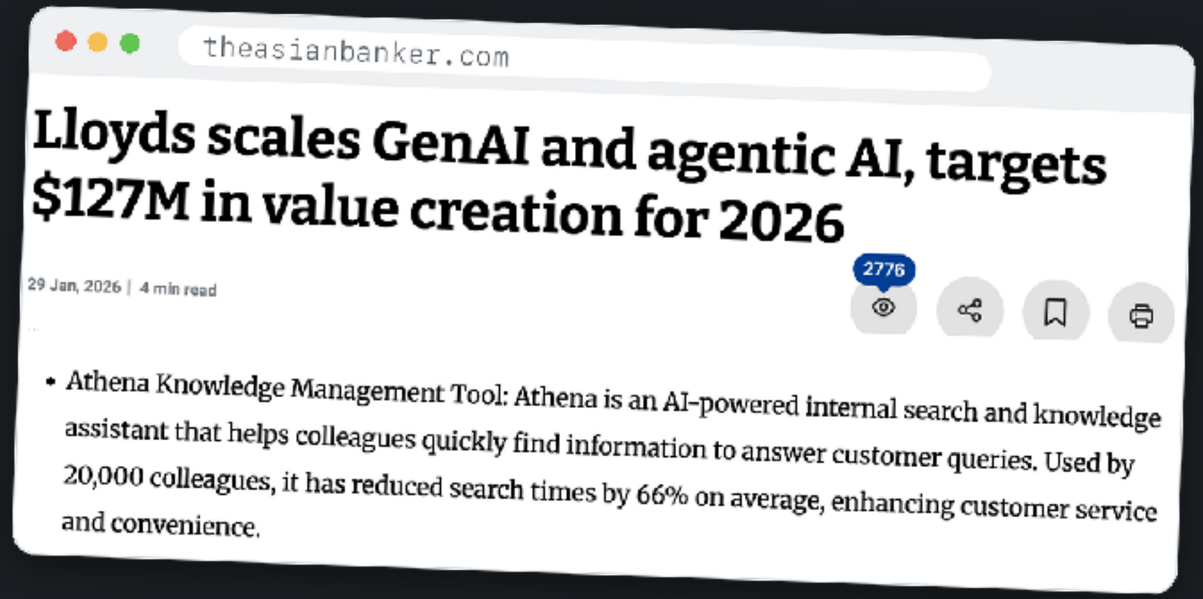
Source: Evident AI Outcomes Report, Aug 2025

2%

of AI use cases in UK finance decide fully on their own

Source: Bank of England and FCA survey, Nov 2024

1 · Read



Lloyds

Less search time for 20,000 colleagues

66%

1 · Read

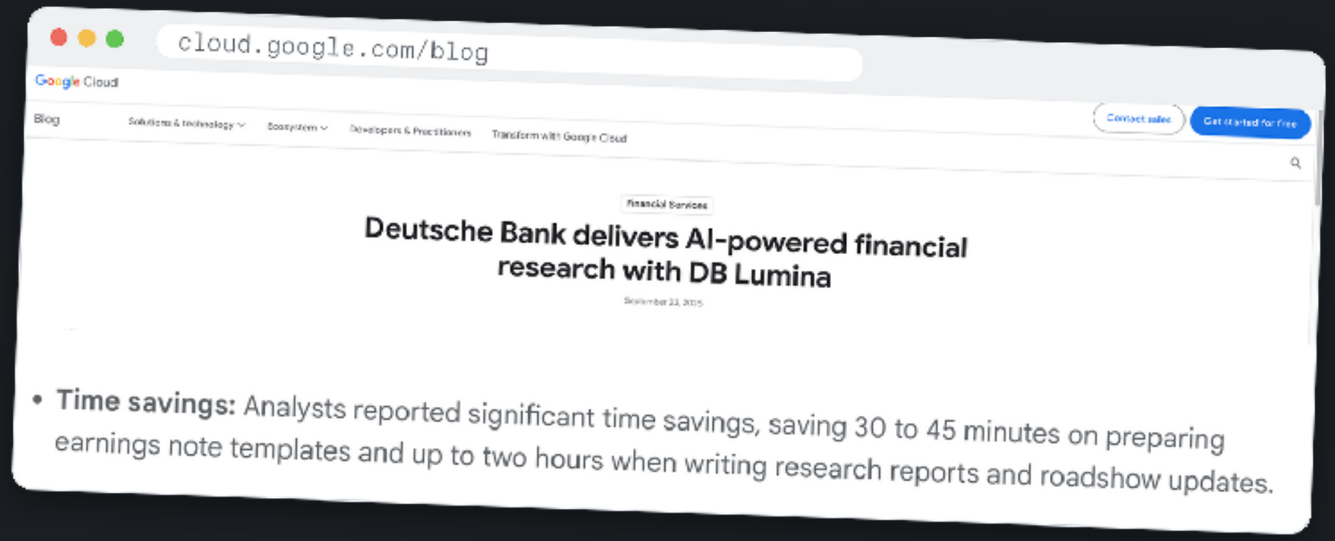


Bank of America

Of staff use Erica for Employees

90%

2 · Draft

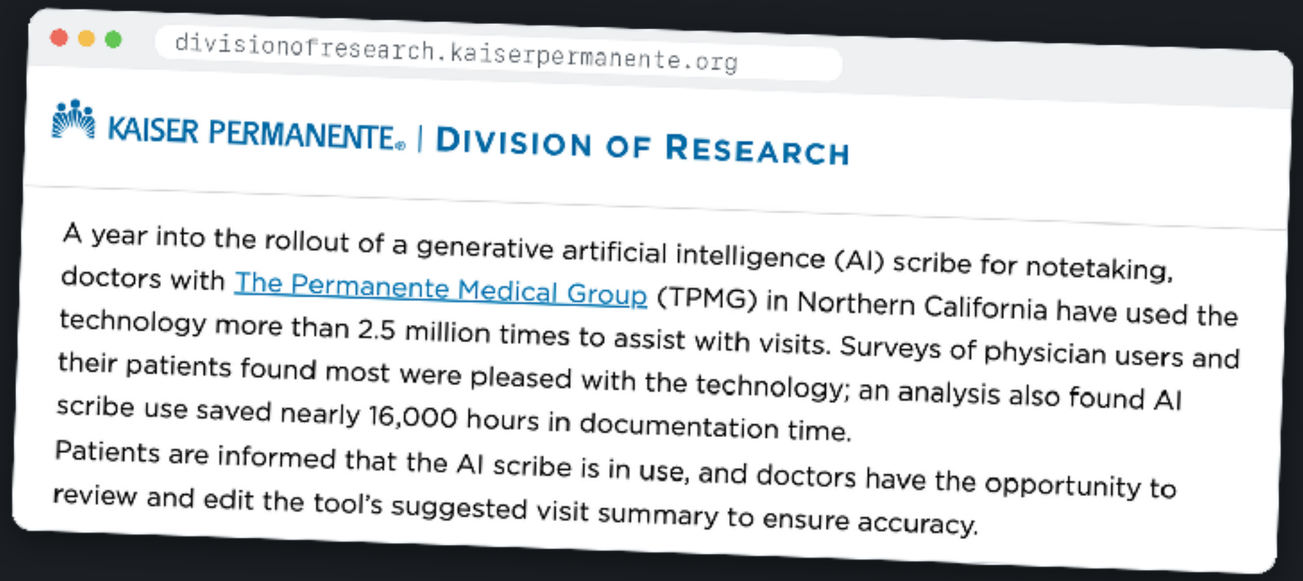


Deutsche Bank

2 hrs

Up to this much saved per research report

2 · Draft



Kaiser Permanente

16K

Hours of note-writing saved in 15 months

3 · Suggest



Westpac

Hours saved by lending agents

150K

4 · Act

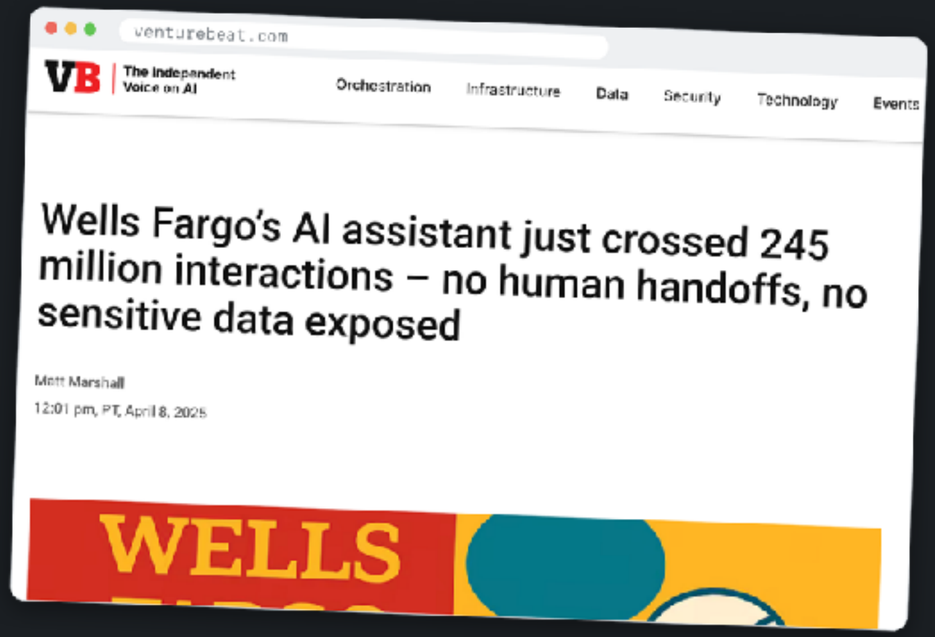


Santander

Faster card fraud claims in Brazil

95%

4 · Act

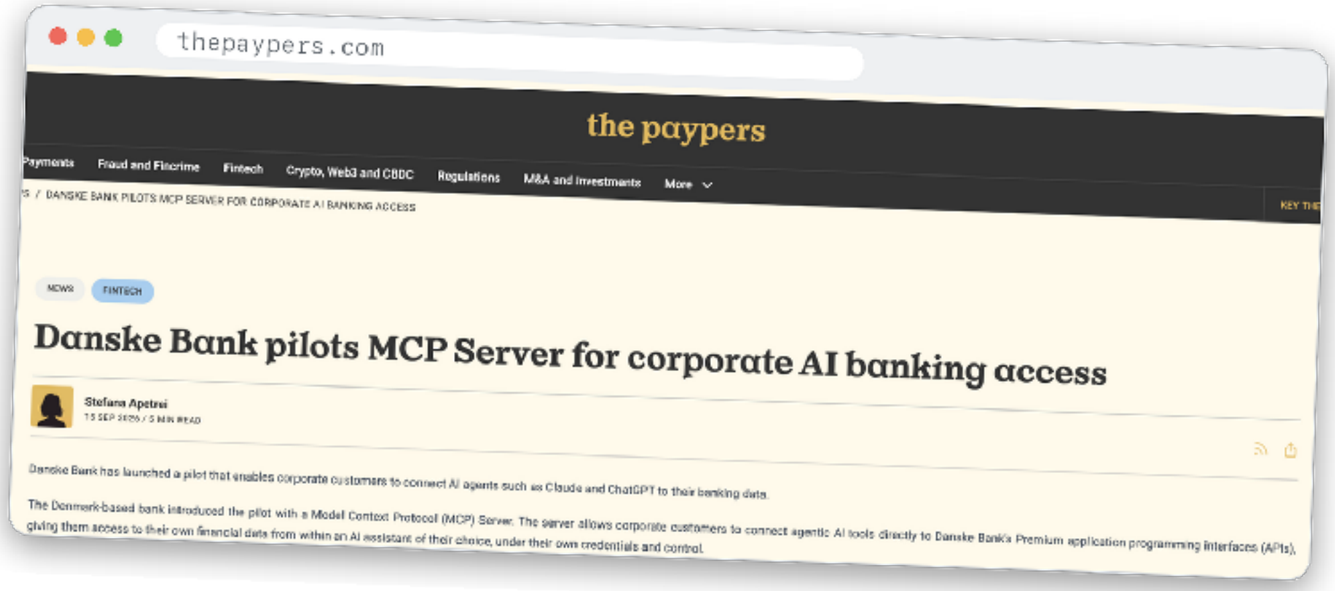


Wells Fargo

245M

Fargo assistant interactions in 2024

Next step



Danske Bank

Clients' own AI agents use its existing APIs

MCP

Skipping steps

What happens when you skip steps

- Commonwealth Bank cut 45 jobs for a voice bot, then reversed
- Air Canada: you own what your bot says
- The CFPB: a bad chatbot can break the law
- Replit: an agent deleted a production database

Your regulator

What changed in 2026

- SR 26-2 replaced SR 11-7 in April 2026
- Gen AI is outside its scope, for now
- EU AI Act chatbot rules apply since August 2026
- Credit scoring is high-risk from December 2027

BMW's rule

Prepared by AI, decided by people.

Getting started

Hire your first intern

- [x] Pick one boring, internal job
- [x] Start read-only: tools that only read
- [x] Test it like code: evals, logs, traces
- [ ] Add drafts, then actions behind a gate
- [ ] Spring AI has a piece for every step

Every one of these is a Java problem.



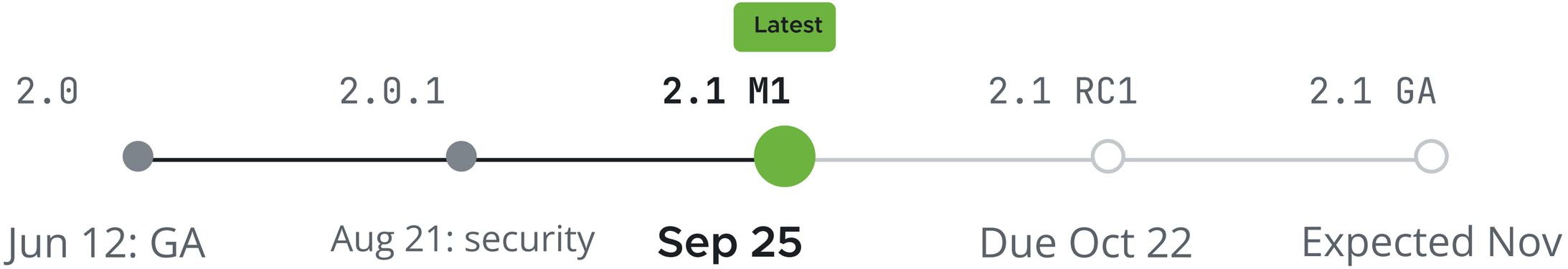
Part two

02

What's new in Spring AI 2

What's new

Four months of releases



1.x

Open source support for Spring AI 1.x ended June 30, 2026

2.0.x is supported until July 2027. Source: Spring support API

1.1.9

The latest Spring AI 1.1 patch is only in Spring Enterprise

Open source stops at 1.1.8. Source: Spring support API, Oct 2026

TrueSource by Broadcom

When open source support ends

Stay exposed	Patch your line Pick	Rush a major
No new fixes	Spring Enterprise	Big upgrade
Findings pile up	Same release line	Lots of testing
Risk grows	Advisor applies it	Not this quarter

TrueSource by Broadcom

TrueSource Spring Enterprise

- Secure builds and 24x7 support for Spring, OpenJDK and Tomcat
- Day-zero CVE-only patches for Spring and its managed dependencies
- Application Advisor applies them, even in CI as pull requests
- Built by the engineers who maintain Spring



What's new

The new baseline

Moved with Boot 4

Spring Boot 4	4.0 and 4.1
Framework 7	Jakarta EE 11
Jackson 3	tools.jackson
JSpecify	Null safety
Options	Immutable

Didn't move

17

Java 17 is still the baseline.
Even for 2.1.

Part two

Tool calling

Tool calling

Tools give the model context or actions

Context

Read something

Data the model doesn't have

recentTransactions

findDuplicateCharges

Actions

Do something

A change in your app

proposeDispute

A person confirms the risky ones

Tool calling

Without a framework, you write the loop

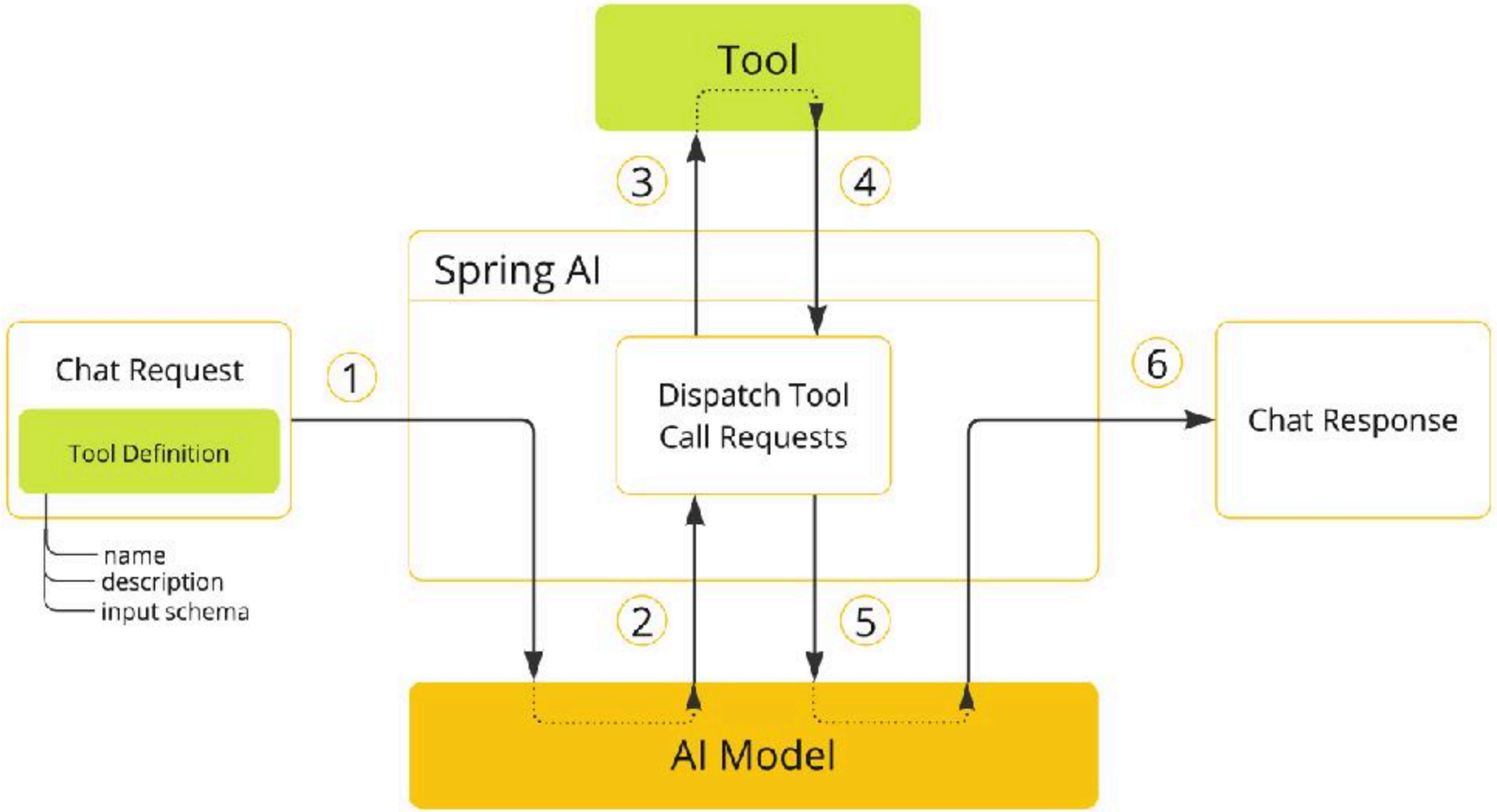
by hand (pseudocode)

```
var reply = model.call(prompt);
while (reply.wantsTool()) {
    var result = run(reply.tool());
    prompt.add(reply, result);
    reply = model.call(prompt);
}
return reply.text();
```

Spring AI runs this loop for you

Tool calling

How a tool call works



2.0

A new tool loop

Tool calling moved into the advisor chain.
One place to see and control every call.

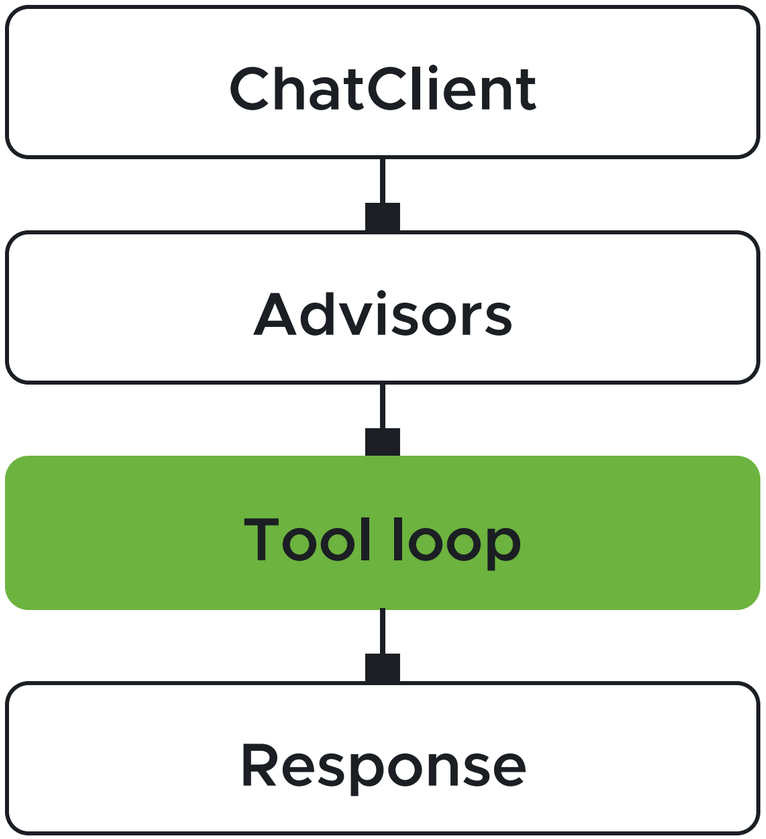
`advisor chain` `auto`

AssistantController.java

```
chatClient.prompt()
    .user(message)
    .tools(customerTools)
    .call()
    .content();
```

New in 2.0

One tool loop, for every provider



Your prompt goes in

Memory, guardrails, logging

ToolCallingAdvisor calls the model and your tools

Every tool call is a traced span

New in 2.0.1

2.0.1 is a security release

7

CVEs fixed

40

Calls per tool, at most

150

Tool calls in total, at most

New default in 2.0.1

Only the tools you attach to a request can run.

Part two

Tool Search

New in 2.0, opt-in

The Tool Search Tool

1

Index

All your tools

2

Search

Model asks for one

3

Expand

Matches added

4

Call

A normal tool call

64%

fewer tokens with the Tool Search Tool, at best

Spring's own benchmark: 34 to 64%

Part two

Structured output

ChatClient

What a call can give you back

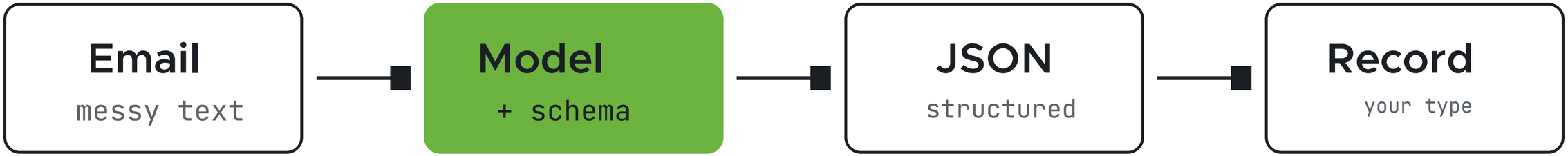
```
chatClient.prompt().user(question)

  .call().content()                    // String
  .call().chatResponse()               // + tokens, model
  .call().entity(DisputeCase.class)    // your record
  .call().responseEntity(...)          // record + metadata
  .call().chatClientResponse()         // + advisor context
  .stream().content()                  // Flux<String>
```

Structured output starts here

Structured output

Text in, a Java object out



Structured output

Ask for a record, not a string

DisputeEmailController.java

```java
// The type you want back
record DisputeCase(String merchant,
    BigDecimal amount) {}

// One call returns it
DisputeCase d = chatClient.prompt()
    .user(email).call()
    .entity(DisputeCase.class);
```

Your record becomes the schema

The catch

The model can still send back JSON that doesn't fit.

New in 2.0

Structured output that checks itself

DisputeEmailController.java

```
DisputeCase dispute = chatClient.prompt()
    .user(email).call()
    .entity(DisputeCase.class, spec → spec
        .validateSchema()
        .useProviderStructuredOutput());
```

New in 2.0 and 2.0.1

See what your AI costs

- Every tool call is an execute_tool span
- Cache hits reported for every provider
- Token use added up across the whole tool loop
- Prompt logging is off unless you turn it on

Part two

Memory

Memory

How AI memory works

- The model remembers nothing. It only sees this prompt.
- Memory is your code putting text back into the prompt.
- Short-term: recent messages from this conversation.
- Long-term: facts saved somewhere, brought back later.

Memory

Short-term vs long-term memory

Today

Short-term

The current conversation

ChatMemory keeps recent messages

JDBC, Redis, Neo4j, Mongo, Cassandra

Later

Long-term

Facts that last across sessions

VectorStoreChatMemoryAdvisor (core)

AutoMemoryTools (community)

Memory

Long-term memory answers two questions

Save

Who saves it?

Your code, or the model?

Advisor: your code saves every message

AutoMemoryTools: the model decides

Recall

How does it come back?

Always added, or looked up?

Advisor: your code adds close matches

AutoMemoryTools: the model looks it up

New in 2.0

Memory changes

- A conversation ID is now required
- Redis joins the memory stores
- Memory saves the final exchange, not the whole tool loop
- Planned for 2.1: sessions that summarize old messages

Part two

New in 2.1

New models

How do we use these in Spring AI?

Flagship models

Start with GPT-6 Astra for complex reasoning and coding, choose GPT-6.1 Sol to balance intelligence and cost, or use GPT-6 Luna for cost-sensitive, high-volume workloads.

View all | Compare models



GPT-6 Astra

Our most capable model for the most demanding work.

Model ID	`gpt-6-astra`
Reasoning	`low` `medium` `high` `xhigh` `max`
Input price	$10 / Input MTok
Output price	$50 / Output MTok



GPT-6.1 Sol

Near-Astra performance for complex work at a lower cost.

Model ID	`gpt-6.1-sol`
Reasoning	`low` `medium` `high` `xhigh` `max`
Input price	$2 / Input MTok
Output price	$10 / Output MTok



GPT-6 Luna

Our most efficient model for focused, high-volume tasks.

Model ID	`gpt-6-luna`
Reasoning	`none` `low` `medium` `high` `xhigh` `max`
Input price	$0.1 / Input MTok
Output price	$0.5 / Output MTok

Why it matters here

GPT-6 Astra and 6.1 Sol need the Responses API to call tools.

New in 2.1.0-M1

The Responses API is one property

application.yaml | application.properties

```yaml
spring:
  ai:
    openai:
      api-key: ${OPENAI_API_KEY}
      chat:
        api: responses
      responses:
        model: gpt-6-astra
```

2.1.0-M1

Message parts

Before: text, with media, tool calls and reasoning on the side.

Now: one ordered list of parts. Text, reasoning, tool calls and media.

New in 2.1.0-M1

Send parts in, in order

UserMessage

```
UserMessage question = UserMessage.builder()
        .part(TextPart.of("Same charge?"))
        .part(MediaPart.of(receipt))
        .part(MediaPart.of(statement))
        .build();
```

Text and images, in the order you add them

New in 2.1.0-M1

Read the parts of a reply

AssistantMessage

```
var reply = response.getResult().getOutput();
for (MessagePart part : reply.getParts()) {
    switch (part) {
        case ReasoningPart r → log(r.summary());
        case ToolCallPart t → log(t.toolCall());
        case TextPart t → log(t.text());
        default → { }
    }
}
```

Reasoning, tool calls and text, in order

DEMO



A banking assistant

Read-only tools, and the customer confirms

```
>_ git switch 2a-assistant
```

DEMO

Read a dispute email

An email in, a typed DisputeCase out

```
>_ git switch 2b-structured-output
```

DEMO

Memory that lasts

Short-term per chat, long-term in files

```
>_ git switch 2c-memory
```

New in 2.0

Provider quick hits

- Anthropic: the official SDK, with Skills, web search and thinking
- OpenAI: one module, and it covers Azure OpenAI
- Fewer providers: check yours before you upgrade
- Embeddings: bring your own with upsert() (2.1)

Upgrading from 1.x

Five steps to 2.x

- ✅ Move to Boot 4 and Jackson 3
- ✅ Call tools through ChatClient, not ChatModel
- ✅ Attach tools to each request
- ✅ Use immutable options and conversation IDs
- ✅ Run Spring Application Advisor

What's next

Coming soon

1

RC1

Parts everywhere

2

2.1 GA

November

3

Session

Across turns

4

Agents

Announced



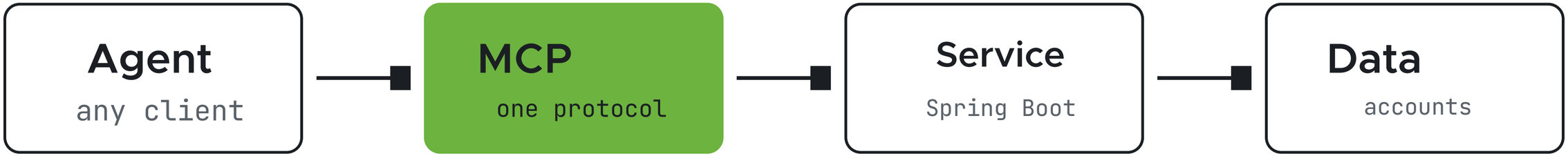
Part three · MCP

03

Your services, ready for agents

MCP

Write the integration once



MCP

MCP grew up

- Moved to the Linux Foundation in December 2025
- OpenAI, Google, Microsoft and AWS all support it
- Wells Fargo and Visa joined the foundation
- The July 2026 spec made it stateless

MCP in Java

The official MCP Java SDK came from the Spring team.

“

APIs are for systems.
MCP is for LLMs.

MCP

An API vs. MCP tools

API

Built for programs

Many endpoints: CRUD, IDs, paging

Docs written for people

One call is rarely the whole task

MCP

Built for models

A few task-shaped tools

Descriptions written for a model

Reads and writes marked, least privilege

Connect

Uber

800+

MCP servers from existing APIs. Every tool starts off.

MCP

Who else runs it

Stripe

A person confirms refunds and payments

Bloomberg

Market data for clients' agents

Visa

Agents reach its payment APIs

MCP in Spring AI 2

Design tools for agents

BankMcpTools.java

```
@Component
public class BankMcpTools {
    @McpTool(description = """
            Find disputes by status, at or
            above an amount in dollars""")
    public DisputeList findDisputes(DisputeStatus status,
            BigDecimal minAmount) {
        return disputeService.findDisputes(status, minAmount);
    }
}
```

Written for a model, not a person

New in 2.0

What's new for MCP in Spring AI 2

- Annotations in core: @McpTool, @McpResource, @McpPrompt
- MCP Java SDK 2.0, for the November 2025 spec
- Streamable HTTP is the way forward. SSE is deprecated.
- MCP tools get the same tool loop, limits and spans

MCP in Spring AI 2

Always set the protocol

application.yaml | application.properties

```yaml
spring:
  ai:
    mcp:
      server:
        name: bank
        protocol: STREAMABLE
```

Security

An MCP endpoint is an API. Secure it like one.

MCP security

What goes wrong, and the fix

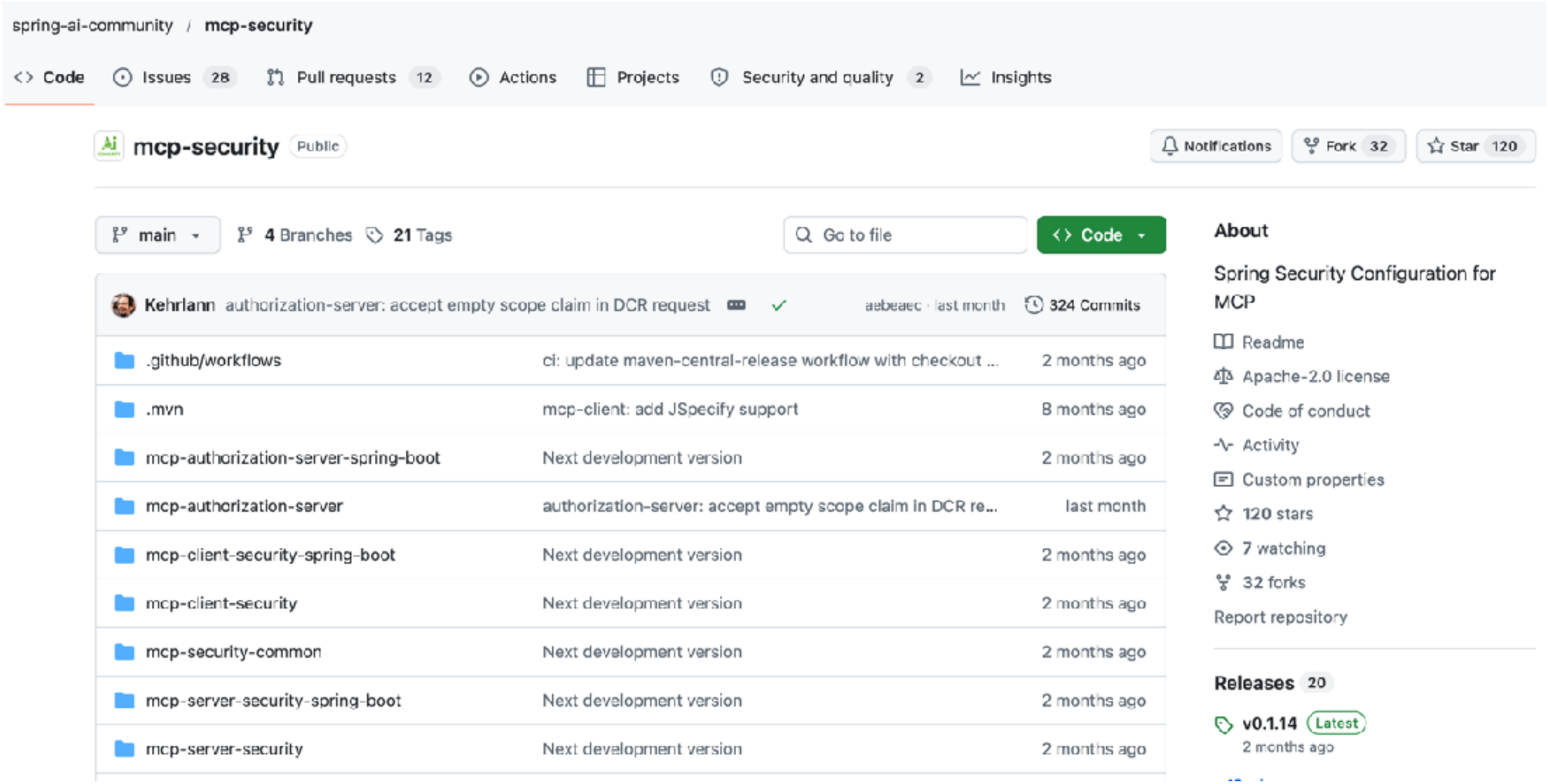
- The lethal trifecta: private data, untrusted input, a way out
- Least privilege on every token
- Remote, approved servers only
- A person confirms every write

In Spring AI 2

A bare MCP server has no auth. Add mcp-security.

Spring AI Community

mcp-security secures your MCP server



DEMO



Your bank over MCP

The same services, used from Claude Code

```
>_ git switch 3-mcp
```



Part four

04

A new kind of model

A new kind of model

Thinking, fast and slow

System 2

Slow and deliberate

Reasons step by step

Writes its answer out

LLMs with reasoning

System 1

Fast and intuitive

A judgment in a second

A typed answer with a confidence

Decision models, like Jev

A new kind of model

LLMs write. Decision models decide.

LLM

Writes

Text in, text out

Seconds per call

You parse the answer

New

Decides

State and typed questions in

Probabilities out

Milliseconds per call

A new kind of model

What decision models are for

Classify

Send an email to the right team

Route models

Cheapest model that can answer

Guard

Block a risky prompt first

Judge

Is it complete? Is it good?

Rank

Keep the docs that matter

Gate

Act, confirm or escalate

A new kind of model

A category in five weeks

1

Sep 15

Jev launches

2

Sep 29

OpenAI, Ollama

3

Oct 1

Clef, Decider

4

Oct 6

OpenAI beta

16×

LLM judges cost at least 16 times more

Up to 357x more, and 5 to 350x slower. Rao and Callison-Burch, arXiv 2609.29769

A new kind of model

What a decision costs

LLM

GPT-6 Astra

$10 per 1M input tokens

$50 per 1M output tokens

Pays for every word it writes

New

Jev

$0.042 per 1M input tokens

No output to pay for

About 240x cheaper on input

Jev

Three kinds of questions

Choice

Pick one option. Which team handles this email?

Returns: the pick, a probability per option, a confidence

Score

Rate it on a rubric. How urgent is this, 1 to 5?

Returns: a score, probabilities, a confidence

Noul

How true is it? Does this prove an address?

Returns: one value from 0 to 1

Spring AI TypeSafe · Community

Typed questions, typed answers

DisputeRouter.java

```java
var team = Choice.builder().instructions("Which team?")
    .option("disputes", "Card charges and refunds")
    .option("fraud", "Stolen cards and scams").build();
var r = typeSafeClient.systemOne(SystemOneRequest.builder()
    .state(email).question("team", team).build());
r.choiceValue("team");            // "disputes"
r.choice("team").confidence();    // 0.91
```

Spring AI TypeSafe · 0.4.0

The building blocks

Judge
Grade answers

Guardrail
Block bad input

Filter
Clean RAG docs

Rerank
Best docs first

Tool index
Pick the tool

Gate
Act or escalate

The confidence gate

Confidence decides if software acts alone.

TransferAssistant.java

```java
var gate = JevConfidenceGate.builder()
        .floor(0.60)
        .require("transfer_funds", 0.85)   // costs more when wrong
        .build();

switch (gate.decide(response.choice("intent"))) {
    case EXECUTE  → perform(intent);
    case CONFIRM  → askTheCustomerToConfirm(intent);
    case ESCALATE → handOverToAPerson();
}
```

Keep your data home

Where the decisions run

Jev	Local	OpenAI
Hosted in the US	Laya or Ollama	US or EU residency
ZDR on request	Data stays home	Beta, gpt-6-luna
$0.042 per 1M input	One property	No Spring AI yet

Pick

A new kind of model

Open source options

- Decision models: Laya, Perplexity Decider, Cloudflare Clef
- Routing: vLLM Semantic Router, semantic-router
- Guardrails: Llama Prompt Guard 2
- Reranking: bge-reranker-v2-m3, mxbai-rerank, Qwen3-Reranker

DEMO



Route to the right model

```
>_ git switch 4-jev
```

Jev picks the cheapest model that can answer



Part five

05

Recap and Q&A

Recap

Four things to take home

1

Start small

One boring job, read-only first

2

Upgrade to 2.x

The tool loop, memory, the Responses API

3

Design tools for agents

APIs are for systems. MCP is for LLMs.

4

Let System 1 decide

Fast, cheap, with a confidence

Wrap-up

Resources

Reference docs

`docs.spring.io/spring-ai`

Upgrade notes

`docs.spring.io/spring-ai/reference/upgrade-notes.html`

Demo code

`github.com/danvega/spring-ai-bank`



Scan for the demo code



Thank you

Questions?

dan.vega@broadcom.com @therealdanvega github.com/danvega